# [프로젝트 기획 보고서] Simple Safe

**프로젝트 요약:** LLM 기반(바이브 코딩)으로 자동 생성된 웹 사이트의 보안 취약점을 문맥 기반으로 분석하고, 시큐어 코딩을 제안 및 자동 적용(PR)하는 맞춤형 AI 보안 에이전트 서비스

**팀원 구성**
한재현 20231626 (팀장)
김민서 20231520 (팀원)

## 1. 배경 및 문제 상황

최근 자연어 지시만으로 코드를 생성하는 **바이브 코딩(Vibe Coding)**의 유행으로 누구나 쉽게 웹사이트를 구축할 수 있게 되었습니다. 하지만 LLM은 보안보다 '기능 구현'에 우선순위를 두는 경향이 있어, API 키 하드코딩, DOM 기반 XSS 필터링 누락, 데이터 검증 없는 동적 렌더링 등 심각한 보안 결함을 내포한 코드를 빈번하게 생성합니다. 보안 지식이 부족한 일반 사용자는 이를 인지하지 못한 채 서비스를 배포하여 중대한 사이버 위협에 노출되고 있습니다.

## 2. 핵심 서비스 및 기대 효과

- **원클릭 심층 진단:** 사용자가 GitHub 저장소 링크나 소스 코드를 제공하면, AI가 전체 코드의 문맥과 데이터 흐름을 추적하여 숨겨진 취약점을 찾아냅니다.
- **문맥 기반(Context-Aware) 보안 리포트:** 입력값이 브라우저에 렌더링되는 상세한 경로를 논리적으로 설명하여, 기존 도구의 잦은 오탐(False Positive)을 획기적으로 줄이고 직관적인 리포트를 제공합니다.
- **시큐어 코딩 즉시 패치 (Auto-Remediation):** 취약점이 발견된 코드를 대체할 안전한 방어 코드를 즉석에서 생성하고, 사용자의 GitHub로 즉시 PR(Pull Request)을 전송하여 문제 해결까지의 시간을 단축합니다.

## 3. 기존 SAST 도구와의 기술적 차별성

| 비교 항목 | 기존 정적 분석 도구 (예: SonarQube) | Simple Safe (당사 솔루션) |
|---|---|---|
| **분석 방식** | 규칙(Regex) 기반 및 AST 패턴 매칭 (알려진 취약점만 탐지) | **문맥 기반 (Context-Aware)** 코드 의도 및 데이터 흐름 추론 |
| **오탐률 (False Positive)** | 높음 (실제 안전한 코드도 패턴에 걸리면 무조건 경고 발생) | **낮음** (입력값의 DOM 렌더링 흐름을 파악하여 진짜 위협만 필터링) |

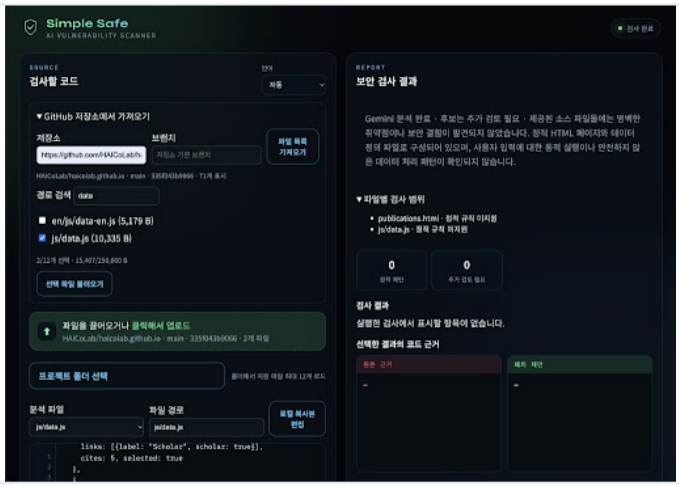
| 비교 항목 | 기존 정적 분석 도구 (예: SonarQube) | Simple Safe (당사 솔루션) |
| --- | --- | --- |
| **해결책 제시** | 수동 수정 (취약점 위치 및 일반적인 가이드라인만 제공) | **자동 패치 및 PR 생성** (문맥에 맞는 방어 코드 즉석 생성) |
| **초기 설정** | 무거움 (별도 서버 구축, CI/CD 파이프라인 연동 필수) | **가벼움 (Zero-Config)** GitHub 링크만으로 즉시 스캔 시작 |
| **핵심 타겟** | 엔터프라이즈 레벨의 사람이 작성한 레거시 코드 | **LLM이 생성한 웹 서비스 (Vibe Coding 결과물) 맞춤형** |

## 4. 개발 및 고도화 과정 (Development Journey)

본 프로젝트는 단순한 아이디어 구상에 머무르지 않고, **'명확한 문제 정의 ➔ 애자일(Agile)한 프로토타입 검증 ➔ 최신 오픈소스 기술의 전략적 통합'**이라는 이상적인 소프트웨어 엔지니어링 프로세스를 거쳐 고도화되었습니다.

### 4.1. 문제 정의 및 초기 프로토타입 구축 (Agile Prototyping)

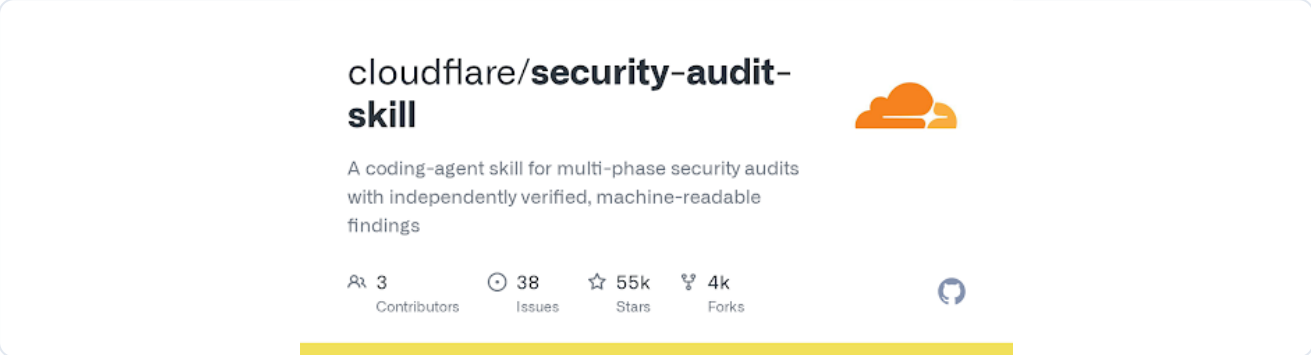
본 프로젝트는 '생성형 AI 기반 웹사이트 구축(Vibe Coding)의 보편화에 따른 보안 취약점 증가'라는 날카로운 문제의식에서 출발했습니다. 기획 의도를 신속하게 검증하기 위해, 팀은 최신 AI 코딩 어시스턴트인 Cursor와 GPT를 활용하여 초기 프로토타입을 즉각 구축했습니다. 이 단계에서는 LLM이 생성한 코드를 분석하고 기본적인 보안 결함을 식별하는 자체 분석 파이프라인을 설계하여 프로젝트의 기술적 실현 가능성을 1차적으로 확인했습니다.




[그림 1] Cursor 및 GPT를 활용하여 구현한 Simple Safe 초기 프로토타입 UI

### 4.2. 오픈소스 생태계를 통한 시장 검증 (Market Validation)

초기 프로토타입 개발이 진행되던 중, 글로벌 오픈소스 플랫폼인 GitHub에 본 프로젝트의 방향성과 정확히 일치하는 목적을 가진 `security-audit-skill`이 공개되었습니다. 이는 우리 팀이 정의한 'LLM 생성 코드의 맞춤형 보안 진단'이라는 의제가 현재 글로벌 테크 씬과 전 세계 개발자 생태계에서도 시급하게 요구되는 핵심 과제임을 입증하는 결정적인 마일스톤이었습니다.




[그림 2] GitHub에 공개된 Cloudflare의 security-audit-skill 레포지토리

### 4.3. 유연한 아키텍처 피벗(Pivot)과 엔진 고도화 (Strategic Integration)

우리 팀은 자체 구축한 초기 엔진에 매몰되는 대신, 시스템의 완성도와 확장성을 극대화하기 위해 유연한 기술적 피벗을 단행했습니다. 시장의 요구에 맞춰 등장한 `security-audit-skill`을 Simple Safe 서비스의 핵심 보안 진단 엔진(Core Engine)으로 전격 교체 및 통합했습니다. 결과적으로 기존 프로토타입 대비 분석의 정확도와 처리 속도가 비약적으로 향상되었으며, 글로벌 스탠다드에 부합하는 보안 규칙을 엣지(Edge) 환경에서 검증할 수 있는 견고한 아키텍처로 진화했습니다.

> **[소결]** Simple Safe 프로젝트는 더 나은 기술적 대안이 등장했을 때 이를 신속하게 수용하고 시스템 아키텍처를 개선하는 유연성을 입증했습니다. 이는 현대 소프트웨어 개발에서 가장 중요하게 평가받는 '문제 해결 능력'과 '오픈소스 활용 역량'을 성공적으로 서비스에 녹여낸 결과입니다.

## 5. 프로젝트 결론 및 향후 발전 방향

### 5.1. 결론 (Conclusion)

**Simple Safe**는 비전문가도 손쉽게 코드를 작성하는 '바이브 코딩' 시대에 필수불가결한 든든한 안전망입니다. LLM이 놓치기 쉬운 문맥 기반의 보안 취약점을 정확히 짚어내고, 즉각적인 수정안(Auto-Remediation PR)을 자동화함으로써 보안 위협을 원천 차단합니다. 본 프로젝트는 최신 AI 기술과 검증된 오픈소스 생태계를 영리하게 결합하여, 실질적이고 즉시 도입 가능한 문제 해결 솔루션을 성공적으로 제시하였습니다.

### 5.2. 향후 발전 방향 (Future Directions)

- **지원 언어 및 환경 확장:** 현재 웹 프론트엔드 중심의 분석에서 나아가 Python, Java 등 백엔드 언어 및 최신 프레임워크(React, Next.js 등)에 대한 맞춤형 보안 진단 기능 확장.
- **DevSecOps 파이프라인 네이티브 통합:** GitHub Actions 및 GitLab CI 등과의 기본 연동(Native Integration)을 지원하여, 개발자의 코드 푸시와 동시에 보안 검수 및 자동 PR이 생성되는 무결점 CI/CD 환경 구축.
- **자체 위협 인텔리전스 구축:** 분석된 취약점 데이터와 패치 사례를 지속적으로 축적하여, LLM 모델을 자체적으로 파인튜닝(Fine-Tuning)함으로써 오탐률을 제로에 가깝게 최소화.